

Task 1: Producer-Consumer using POSIX semaphores

Code:

```
karthik@Karthikeya: ~/ossp/v x + v
GNU nano 7.2 producer_consumer.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
#define ITEMS 10

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
}

void *consumer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify  ^/ Go To Line M-E Redo     M-6 Copy
```

Output:

```
karthik@Karthikeya: ~/ossp/v  x  +  v
karthik@Karthikeya:~$ mkdir -p ~/ossp/week12
karthik@Karthikeya:~$ cd ~/ossp/week12
karthik@Karthikeya:~/ossp/week12$ nano producer_consumer.c
karthik@Karthikeya:~/ossp/week12$ gcc producer_consumer.c -o producer_consumer -pthread
karthik@Karthikeya:~/ossp/week12$ ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
```

Task 2: Deadlock demonstration using two mutexes

Code:

```
karthik@Karthikeya: ~/ossp/v  ×  +  ~
GNU nano 7.2                                deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg) {
    pthread_mutex_lock(&lock1);
    printf("Thread 1 acquired lock 1\n");

    sleep(1);

    printf("Thread 1 waiting for lock 2\n");
    pthread_mutex_lock(&lock2);

    printf("Thread 1 acquired lock 2\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void *thread2(void *arg) {
    pthread_mutex_lock(&lock2);

    printf("Thread 2 acquired lock 2\n");

    sleep(1);

    printf("Thread 2 waiting for lock 1\n");
    pthread_mutex_lock(&lock1);

    printf("Thread 2 acquired lock 1\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^/_ Go To Line M-E Redo     M-6 Copy
```

Output:

```
karthik@Karthikeya:~/ossp/week12$ nano deadlock.c
karthik@Karthikeya:~/ossp/week12$ ooo
Command 'ooo' not found, did you mean:
  command 'roo' from snap roo (2.0.3)
  command 'oor' from deb openoverlayrouter (1.3.0+ds1-3)
  command 'odoo' from deb odoo-16 (16.0.0+dfsg.2-2)
  command 'goo' from deb goo (0.155+ds-4)
See 'snap info <snapname>' for additional versions.
karthik@Karthikeya:~/ossp/week12$
karthik@Karthikeya:~/ossp/week12$ gcc deadlock.c -o deadlock -pthread
timeout 5s ./deadlock
Thread 1 acquired lock 1
Thread 2 acquired lock 2
Thread 1 waiting for lock 2
Thread 2 waiting for lock 1
```

Task 3: Prevent deadlock using resource ordering

Code:

```
karthik@Karthikeya: ~/ossp/v  ×  +  ▾
GNU nano 7.2                                deadlock_prevention.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg) {
    int id = *(int *)arg;

    pthread_mutex_lock(&lock1);
    printf("Thread %d acquired lock 1\n", id);

    pthread_mutex_lock(&lock2);
    printf("Thread %d acquired lock 2\n", id);

    printf("Thread %d is working\n", id);
    sleep(1);

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    printf("Completed without deadlock.\n");
    return 0;
}

⌘G Help      ⌘O Write Out  ⌘W Where Is  ⌘K Cut        ⌘T Execute   ⌘G Location  ⌘U Undo      ⌘-A Set Mark  ⌘-I To Bracket
⌘X Exit      ⌘R Read File  ⌘_ Replace   ⌘V Paste     ⌘J Justify   ⌘/ Go To Line ⌘-E Redo     ⌘-G Copy     ⌘-Q Where Was
```

Output:

```
karthik@Karthikeya:~/ossp/week12$ nano deadlock_prevention.c
karthik@Karthikeya:~/ossp/week12$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
./deadlock_prevention
Thread 1 acquired lock 1
Thread 1 acquired lock 2
Thread 1 is working
Thread 2 acquired lock 1
Thread 2 acquired lock 2
Thread 2 is working
Completed without deadlock.
karthik@Karthikeya:~/ossp/week12$
```